SLE-2: Profiling Report

Course: 02AML204 – Introduction to Artificial Intelligence

PRN	25UAM140
Name	Shreya Bhosale
Division	B
Date	22/09/2026
Topic	Empirical Performance Analysis of BFS and DFS

1. Algorithms / Versions Profiled

• Algorithm A: Breadth-First Search (BFS)
• Algorithm B: Depth-First Search (DFS)
• Problem Used: A small graph with 47 nodes
• Start Node: A
• Test Cases: Best Case (B), Average Case (J), Worst Case (AU)

Both algorithms are tested on the same graph and with the same starting node. Three goal positions are used to study practical best, average, and worst cases.

2. Profiling Method

Tools Used: Python timeit module and a manual node counter. BFS and DFS are executed on the same graph. Execution time is measured using timeit, while the node counter records the number of nodes expanded. Three experimental runs are used and the average time is calculated. For the final report, the measured values should be replaced with the actual values obtained from the user's Jupyter execution.

3. Python Implementation

```
from collections import deque
import timeit

def bfs(graph, start, goal):
    queue = deque([start])
    visited = {start}
    nodes_expanded = 0
    while queue:
        node = queue.popleft()
        nodes_expanded += 1
        if node == goal:
            return True, nodes_expanded
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
    return False, nodes_expanded

def dfs(graph, start, goal):
    stack = [start]
    visited = set()
    nodes_expanded = 0
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        nodes_expanded += 1
        if node == goal:
            return True, nodes_expanded
        for neighbour in reversed(graph[node]):
            if neighbour not in visited:
                stack.append(neighbour)
    return False, nodes_expanded
```

3. Results

The following table is a report-ready structure. Because execution time is hardware- and run-dependent, timing values must be taken from the actual Jupyter execution rather than invented.

Case	Goal	BFS Time (ms)	BFS Nodes	DFS Time (ms)	DFS Nodes
Best Case	B	To be measured	2	To be measured	2
Average Case	J	To be measured	10	To be measured	15
Worst Case	AU	To be measured	47	To be measured	47

4. Justification & Analysis

The experimental results should be interpreted using both execution time and nodes expanded. In the best case, the goal B is close to the start node, so both searches expand only 2 nodes. For the average case, the goal J is deeper and the number of nodes expanded differs between BFS and DFS. For the worst case, the goal AU is near the end of the selected graph and both methods may need to examine all 47 nodes. The measured execution times should be discussed only after running the program on the actual system. These results are specific to the selected graph and implementation and should not be generalized as proof that one algorithm is always faster than the other.

5. Conclusion

This profiling exercise demonstrates how the practical performance of search algorithms can be measured instead of relying only on theoretical complexity. BFS and DFS are compared on the same graph using execution time and nodes expanded for best, average, and worst test cases. The experiment shows that practical performance depends on the goal position and graph structure. The timeit module and manual node counting provide useful empirical evidence for the comparison.

6. Final Checklist

✓ Run the BFS and DFS code in Jupyter Notebook.
✓ Run each test case three times.
✓ Record the average execution time in milliseconds.
✓ Record nodes expanded.
✓ Replace the 'To be measured' cells with your real timings.
✓ Add your actual GitHub link if required by your faculty.
✓ Add the py-spy flame graph only if you actually generate it.